

1) ARITHMETIC OPERATORS:

```
# Read two integers from input
a = int(input())
b = int(input())

# Print sum
print(a + b)

# Print difference
print(a - b)

# Print product
print(a * b)
```

2) STRING SIMILARITY

```
def calculate_similarity(s):
    n = len(s)
    z = [0] * n
    l, r = 0, 0
    for i in range(1, n):
        if i <= r:
            z[i] = min(r - i + 1, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] - 1 > r:
                l, r = i, i + z[i] - 1
    return sum(z) + n # Include full string similarity

# Read input
t = int(input())
for _ in range(t):
    s = input().strip()
    print(calculate_similarity(s))
```

3) EXCEPTIONS

```
# Read number of test cases
t = int(input())

# Process each test case
for _ in range(t):
    try:
        a, b = input().split()
        result = int(a) // int(b)
        print(result)
    except (ZeroDivisionError, ValueError) as e:
        print("Error Code:", e)
```

4) ARRAY MATHEMATICS

```
import numpy as np

# Read dimensions
N, M = map(int, input().split())

# Read array A (N lines)
A = np.array([list(map(int, input().split())) for _ in range(N)])

# Read array B (N lines)
B = np.array([list(map(int, input().split())) for _ in range(N)])

# Perform operations and print results
print(A + B)
print(A - B)
print(A * B)
print(A // B)      # Integer (floor) division
print(A % B)
print(A ** B)
```

5) SIMPLE GAME SIMULATION USING PYGAME

```
import pygame
import sys

# Initialize Pygame
pygame.init()

# Screen setup
WIDTH, HEIGHT = 600, 400
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Simple Pygame Application")

# Colors
WHITE = (255, 255, 255)
BLUE = (0, 0, 255)
RED = (255, 0, 0)

# Player and Enemy setup
player = pygame.Rect(50, 300, 40, 40)
enemy = pygame.Rect(100, 100, 40, 40)
enemy_speed = 3
player_speed = 5

# Font for Game Over
font = pygame.font.SysFont(None, 55)

# Game variables
game_over = False
clock = pygame.time.Clock()

# Game loop
while True:
    screen.fill(WHITE)
```

```

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()

if not game_over:
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] and player.left > 0:
        player.x -= player_speed
    if keys[pygame.K_RIGHT] and player.right < WIDTH:
        player.x += player_speed
    if keys[pygame.K_UP] and player.top > 0:
        player.y -= player_speed
    if keys[pygame.K_DOWN] and player.bottom < HEIGHT:
        player.y += player_speed

    # Move enemy
    enemy.x += enemy_speed
    if enemy.right >= WIDTH or enemy.left <= 0:
        enemy_speed *= -1

    # Collision detection
    if player.colliderect(enemy):
        game_over = True

    pygame.draw.rect(screen, BLUE, player)
    pygame.draw.rect(screen, RED, enemy)
else:
    text = font.render("Game Over", True, (0, 0, 0))
    screen.blit(text, (WIDTH//2 - 100, HEIGHT//2 - 25))

pygame.display.update()
clock.tick(30)

```